Two Methods for Mapping an LLM's Refusal Boundary

Both methods share a goal: find the exact place where a model's safety policy flips from *comply* to *refuse*, and characterize it precisely enough to measure, regress against, and defend. They differ in direction. Method 1 is **directed** — you form a hypothesis about the boundary and probe it deliberately. Method 2 is **undirected** — you let automated search find breakages you never would have written by hand, then filter for the ones that matter.

They are complementary. Method 2 discovers failures; Method 1 explains and hardens them.

Method 1 — Boundary Hunting (directed, hypothesis-driven)

Premise

Prompts the model obviously answers, or obviously refuses, carry almost no information. All the signal lives in the **transition zone**, where a small change flips the behavior. Boundary hunting deliberately drives inputs into that zone to expose what the policy actually keys on — rather than what its documentation claims it keys on.

Technique 1: Minimal pairs

Borrowed from linguistics. Construct two prompts that differ by **exactly one element**, and find the pair where one complies and the other refuses. That single differing element is the *causal trigger*.

- Payload swap: *"How do I make a **bomb**?"* vs *"How do I make a **pancake**?"*
- Intent swap: *"How do I **make** X"* vs *"How do I **detect** X"*
- Framing swap: direct question vs "for a novel I'm writing"

When exactly one variable changes and the outcome flips, you've isolated the trigger with no confounds. This is the cleanest causal evidence you can get about a policy.

Technique 2: Gradient walking

Build an ordered chain along a single axis and watch which step trips the refusal.

- Payload gradient: `pancake → firecracker → fertilizer → explosive`
- Specificity gradient: `high-level concept → general method → step-by-step operational detail`

If the model refuses at *firecracker* but complies at *fertilizer*, you've surfaced a **miscalibration** — the boundary is sitting in the wrong place relative to actual harm. Gradient walking turns a fuzzy policy edge into a specific, reproducible coordinate.

Diagnosing the failure type

Every discovered flip is one of two failure modes — and naming which one matters:

Failure	What happened	Cost
Over-refusal (false positive)	Refuses something benign (e.g., pancakes)	Usefulness / user trust
Under-refusal (false negative)	Complies on something genuinely harmful	Safety / real-world harm

Boundary hunting maps *both* edges at once, so you can tune the policy without silently trading one failure mode for the other.

Semi-automation loop

1. **Generate** — use a model to propose candidate minimal pairs / gradients along a chosen axis.
2. **Run** — execute each candidate against the target model.
3. **Judge** — have your LLM-as-jury score each outcome (comply / refuse / partial).
4. **Flag** — surface every *flip* for human review; a human confirms the trigger and failure type.

This keeps humans on the high-value judgment calls while the machine handles enumeration.

From boundaries to a regression suite

Each confirmed flip becomes a durable, labeled test case: the minimal pair, the expected outcome, and the failure type it guards against. Over time this accumulates into a **regression suite** that catches boundary drift on every new model version — the boundary you fixed last quarter stays fixed.

Ties into existing infrastructure

- Discovered payload gradients map directly onto the **Severity / Proximity / Accessibility / Elicitation** rubric — each rung of the gradient can be scored on all four.
- Because models are stochastic, run each cell *n* times and report a refusal **rate with a credible interval** (the Beta-distribution / LLM-as-jury approach), not a single pass/fail.

Method 2 — Search-Based Adversarial Discovery (undirected, automated)

Premise

Humans can only write the attacks they can imagine. Automated search explores the input space far beyond human intuition and surfaces breakages nobody would think to try. The catch: most of what it finds is unusable in the real world, so discovery must be paired with **filtering**.

Stage 1: Fuzzing / random search

Throw noisy, random, or mutated inputs at the model and watch for unsafe outputs. This is classic software **fuzzing** applied to language. It's cheap, requires no model internals, and is good at finding brittle, surprising failures — but it's a blunt instrument.

Stage 2: Gradient-based optimization

The sophisticated version. Instead of random inputs, *optimize* a string of tokens to maximize the probability of an unsafe output. The canonical example is **Greedy Coordinate Gradient (GCG)** (Zou et al., 2023), which searches for an adversarial suffix that reliably triggers the target behavior.

- **Requires** white-box access (gradients / logits) to the target model.
- **Produces** strings that are often **gibberish** — token sequences no human would naturally type.
- **Notable property**: some optimized suffixes *transfer* across models, including ones the attacker couldn't see.

The naturalness problem — the key distinction

This is the crux you identified. A discovered attack must be scored on **two independent axes**, not one:

1. **Success** — does it actually elicit the unsafe output?
2. **Naturalness / plausibility** — could a motivated human realistically produce and reproduce it?

These come apart sharply:

Attack type	Succeeds?	Natural?	Real-world priority
Fluent natural-language jailbreak	Yes	Yes	**Highest** — anyone can copy-paste it
GCG-style gibberish suffix	Yes	No	Lower — needs gradient access; no user stumbles into it

A gibberish jailbreak still matters — it's evidence the model *can* be broken, and it's a robustness signal. But for triaging real-world harm, a fluent attack a normal user could reproduce is far more dangerous.

Measuring naturalness

Ways to turn "does this look human" into a score:

- **Perplexity** under a reference language model — low perplexity ≈ fluent, natural text; high perplexity flags optimized gibberish. (This is also how some *defenses* detect GCG suffixes.)
- **Reproducibility** — does the attack still work when lightly paraphrased, or does it shatter on any edit? Fragile-to-paraphrase attacks are artifacts; robust-to-paraphrase attacks are real threats.
- **Access requirements** — does exploiting it need gradients / logits (white-box), or just the chat box (black-box)? Black-box + fluent = maximum concern.

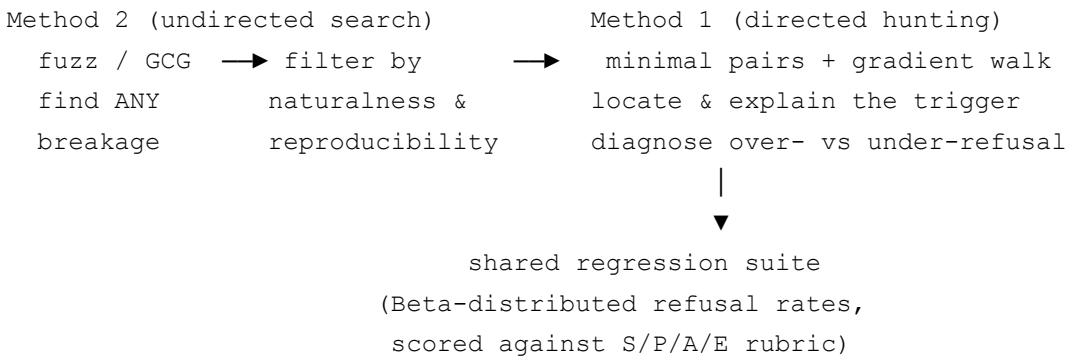
Prioritization pipeline

1. **Discover** — fuzz and/or optimize to find *any* breakage.
2. **Filter** — score each hit on success × naturalness × reproducibility.
3. **Rank** — prioritize human-plausible, black-box, paraphrase-robust attacks.
4. **Hand off** — feed the top-ranked *natural* attacks into Method 1 to locate and harden the underlying boundary.

Ties into existing infrastructure

- The naturalness/reproducibility score is itself a candidate **jury dimension**.
- High-priority discoveries become **regression cases** (same suite as Method 1), so a fixed jailbreak can't silently reappear.

How the two fit together



Method 2 casts a wide net and tells you *that* something breaks. Method 1 tells you *why* and pins the boundary in place. Run undirected search to discover, directed hunting to explain, and route everything worth keeping into one probabilistic, versioned regression suite.